Week 12: Introduction to Simulation

Simulation could be define informally as a computational technique that mimics the behavior of real-world systems or processes through mathematical models.

It involves creating a virtual environment where different scenarios can be explored, allowing analysts to study the effects of various factors and make informed decisions. In simulation, complex systems are broken down into smaller, more manageable components, and their interactions are simulated over time. This approach enables researchers to study the system's behavior under different conditions without the need for costly or time-consuming real-world experiments.

One of the most know simulation methods is known as The Monte Carlo method. A prominent example of simulation that employs random sampling to estimate the probability of different outcomes in a problem, making it a versatile tool across various domains.

Recap on "useful" probability formulas

1. **Probability of the Union**:
   - For two events A and B:

$$P(A \cup B) = P(A) + P(B) - P(A \cap B)$$

2. **Complementary Probability**:
   - Probability of the complement of event A:

$$P(A^c) = 1 - P(A)$$

3. **Conditional Probability**:
   - Probability of event A given event B:

$$P(A|B) = \frac{P(A \cap B)}{P(B)}$$

4. **Intersection for Conditional Probability**:
   - For events A and B:

$$P(A \cap B) = P(B) \times P(A|B)$$

5. **Probability of the Intersection** (When events are independent):
   - For two independent events A and B:

$$P(A \cap B) = P(A) \times P(B)$$

6. **Bayes' Theorem**:

   - For events A and B:

$$P(A|B) = \frac{P(B|A) \times P(A)}{P(B)}$$

7. **Total Law of Probability**:

   - For partitioned events B1, B2, ..., Bn:

$$P(A) = \sum_{i=1}^{n} P(A|B_i) \times P(B_i)$$

8. **Probability using De Morgan's Laws**:

   - De Morgan's Law for unions:

$$P((A \cup B)^c) = P(A^c \cap B^c)$$

   - De Morgan's Law for intersections:

$$P((A \cap B)^c) = P(A^c \cup B^c)$$

Random Variables

Random variables are fundamental concepts in probability theory and statistics, representing numerical outcomes of random phenomena.

A random variable is a mathematical function that assigns a numerical value to each outcome of a random experiment. It represents the uncertain quantities in a probabilistic scenario. Formally, a random variable X is defined as a mapping from the sample space S of a probability space (S, Σ, P) to the real numbers $\mathbb{R}$, i.e., $X : S \to \mathbb{R}$.

Random variables can be either discrete or continuous, depending on whether their possible values form a countable or uncountable set, respectively.

Examples:

1. **Coin Toss**: Let's define a random variable X representing the outcome of a fair coin toss. The sample space S consists of two possible outcomes: heads (H) and tails (T). Thus, $S = \{H, T\}$. We can define X as a mapping from the sample space to the real numbers, $X : S \to \mathbb{R}$, where:

   - If the outcome is heads (H), then $X(H) = 1$.
   - If the outcome is tails (T), then $X(T) = 0$.

   Here, X is a discrete random variable with two possible outcomes, 1 (corresponding to heads) and 0 (corresponding to tails).

2. **Roll of a Six-Sided Die**: Consider a random variable Y representing the outcome of rolling a fair six-sided die. The sample space S consists of the numbers 1, 2, 3, 4, 5, and 6. Thus, $S = \{1, 2, 3, 4, 5, 6\}$. We can define Y as a mapping from the sample space to the real numbers, $Y : S \to \mathbb{R}$, where:

   - If the outcome is 1, then $Y(1) = 1$.
   - If the outcome is 2, then $Y(2) = 2$.
   - If the outcome is 3, then $Y(3) = 3$.
   - If the outcome is 4, then $Y(4) = 4$.
   - If the outcome is 5, then $Y(5) = 5$.
   - If the outcome is 6, then $Y(6) = 6$.

   Here, Y is a discrete random variable with six possible outcomes, each corresponding to one of the numbers on the die.

These examples illustrate discrete random variables where the outcomes are finite and countable, and each outcome has an associated probability.

Computing probabilities from Random Variables

1. **Probability Mass Function (PMF)**: The probability mass function $P(X = x)$ of a discrete random variable X defines the probability that X takes on a specific value x in the sample space S. Formally, it is a function that assigns a probability to each possible outcome in the sample space of X. Mathematically, the PMF is defined as:

$$P(X = x) = \text{probability that } X \text{ takes on the value } x$$

The PMF satisfies the following properties:

- $P(X = x) \geq 0$ for all x in the sample space of X.
- $\sum_x P(X = x) = 1$, where the sum is taken over all possible outcomes in the sample space.

In other words, the PMF describes the probability distribution of a discrete random variable by specifying the likelihood of each possible outcome.

2. **Cumulative Distribution Function (CDF)**: The cumulative distribution function (CDF) of a discrete random variable X, denoted as $F(x)$, gives the probability that X takes on a value less than or equal to x. Mathematically, it is defined as:

$$F(x) = P(X \leq x) = \sum_{t \leq x} P(X = t)$$

Where:

- $F(x)$ is the cumulative probability that X is less than or equal to x.
- $P(X = t)$ is the probability mass function (PMF) of X at the value t.
- The sum is taken over all possible outcomes t less than or equal to x.

The CDF provides a comprehensive view of the probability distribution of a discrete random variable by accumulating the probabilities of all outcomes up to a specified value x.

Example: Throwing a six sided die

Sample Space:

The sample space S represents all possible outcomes of a random experiment. In the case of rolling a fair six-sided die, the sample space consists of the numbers 1, 2, 3, 4, 5, and 6. Therefore, $S = \{1, 2, 3, 4, 5, 6\}$

Random Variable:

In this example, the random variable X represents the outcome of rolling a fair six-sided die. Each outcome x in the sample space corresponds to a value of X. Formally, we define the random variable as:

$X : S \to \mathbb{R}$, where $X(s) = s$.

Probability Mass Function (PMF):

For a fair six-sided die, each outcome has an equal probability of $\frac{1}{6}$. Thus, the PMF is:

$P(X = x) = \frac{1}{6}$ for every $x \in S$.

Cumulative Distribution Function (CDF):

It is calculated by summing the probabilities of all outcomes up to x. For the fair six-sided die, the CDF is:

$F(x) = P(X \leq x) = \sum_{t \leq x} P(X = t)$

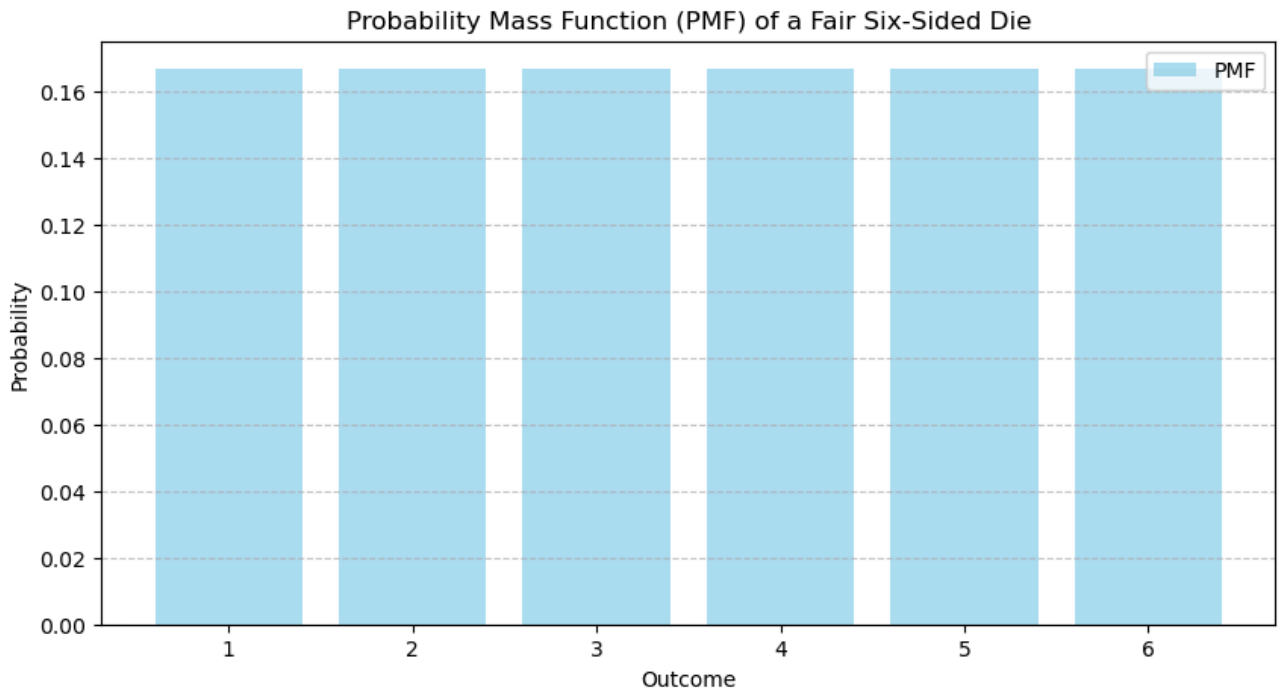
Where:

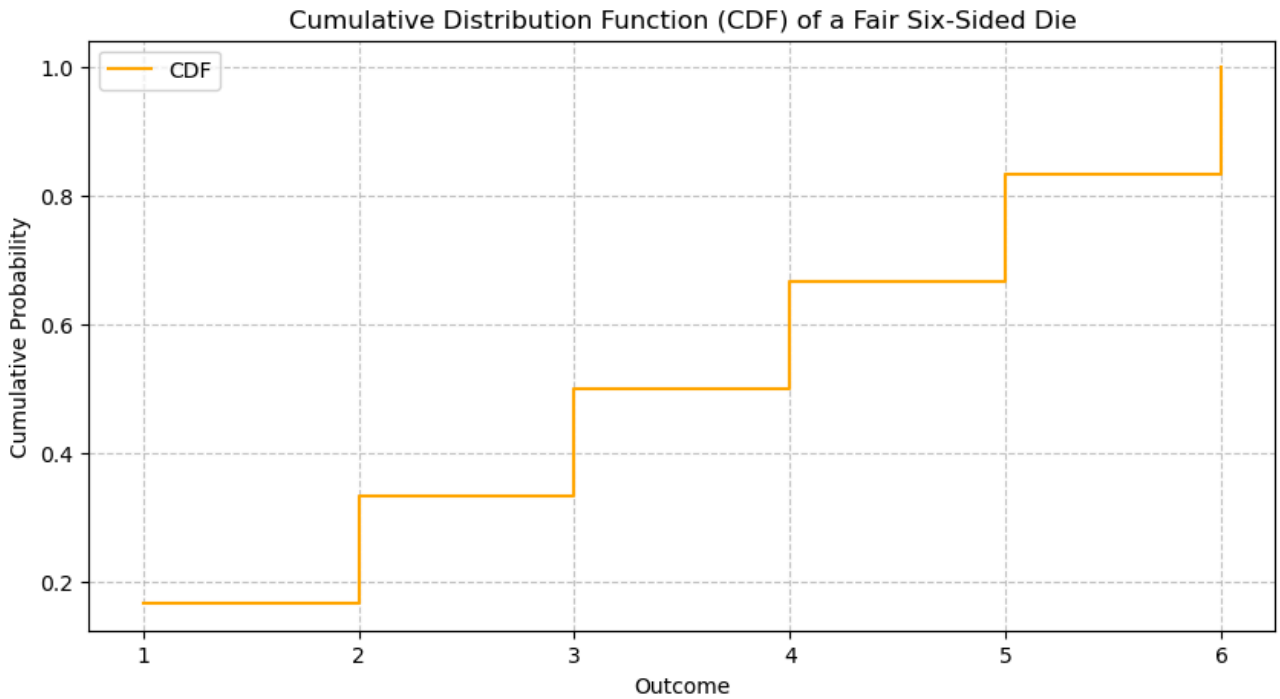
- $F(x)$ is the cumulative probability that X is less than or equal to x.
- $P(X = t)$ is the probability mass function (PMF) of X at the value t.
- The sum is taken over all possible outcomes t less than or equal to x.

Therefore, $F(1) = 1/6$,$F(2) = 1/3$,$F(3) = 1/2$,$F(4) = 2/3$,$F(5) = 5/6$ and $F(6) = 1$.

In [20]:

```
import numpy as np
import matplotlib.pyplot as plt

# Define the sample space and PMF
sample_space = np.array([1, 2, 3, 4, 5, 6])
pmf = np.ones_like(sample_space) / len(sample_space)

# Calculate the CDF
cdf = np.cumsum(pmf)

# Plot the PMF
plt.figure(figsize=(10, 5))
plt.bar(sample_space, pmf, color='skyblue', alpha=0.7, label='PMF')
plt.title('Probability Mass Function (PMF) of a Fair Six-Sided Die')
plt.xlabel('Outcome')
plt.ylabel('Probability')
plt.xticks(sample_space)
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.legend()
plt.show()

# Plot the CDF
plt.figure(figsize=(10, 5))
plt.step(sample_space, cdf, color='orange', where='post', label='CDF')
plt.title('Cumulative Distribution Function (CDF) of a Fair Six-Sided Die')
plt.xlabel('Outcome')
plt.ylabel('Cumulative Probability')
plt.xticks(sample_space)
plt.grid(axis='both', linestyle='--', alpha=0.7)
plt.legend()
plt.show()
```





Discrete Probability Distributions

Probabilistic discrete distributions are mathematical functions that describe the probabilities of different outcomes of a discrete random variable. These distributions provide a framework for understanding the likelihood of various type of events. Here are some of the most well-known probabilistic discrete distributions:

Discrete Uniform Distribution

The Discrete Uniform distribution represents a finite set of equally likely outcomes. Each outcome has the same probability of occurring, and there is a constant probability for each possible

outcome within the range. This distribution is commonly used when modeling situations where each outcome is equally likely.

Probability Mass Function (PMF)

The probability mass function (PMF) of a Discrete Uniform random variable X defined on the set $\{a, a+1, \ldots, b\}$ is given by:

$$P(X = x) = \frac{1}{b - a + 1}$$

Where:

- a is the minimum value of the range.
- b is the maximum value of the range.
- x is any integer in $[a, b]$.

The PMF specifies the probability of each outcome within the range.

Cumulative Distribution Function (CDF)

The cumulative distribution function (CDF) of a Discrete Uniform random variable X is given by:

$$F(x) = \begin{cases} 0 & \text{if } x < a \\ \frac{x-a+1}{b-a+1} & \text{if } a \leq x \leq b \\ 1 & \text{if } x > b \end{cases}$$

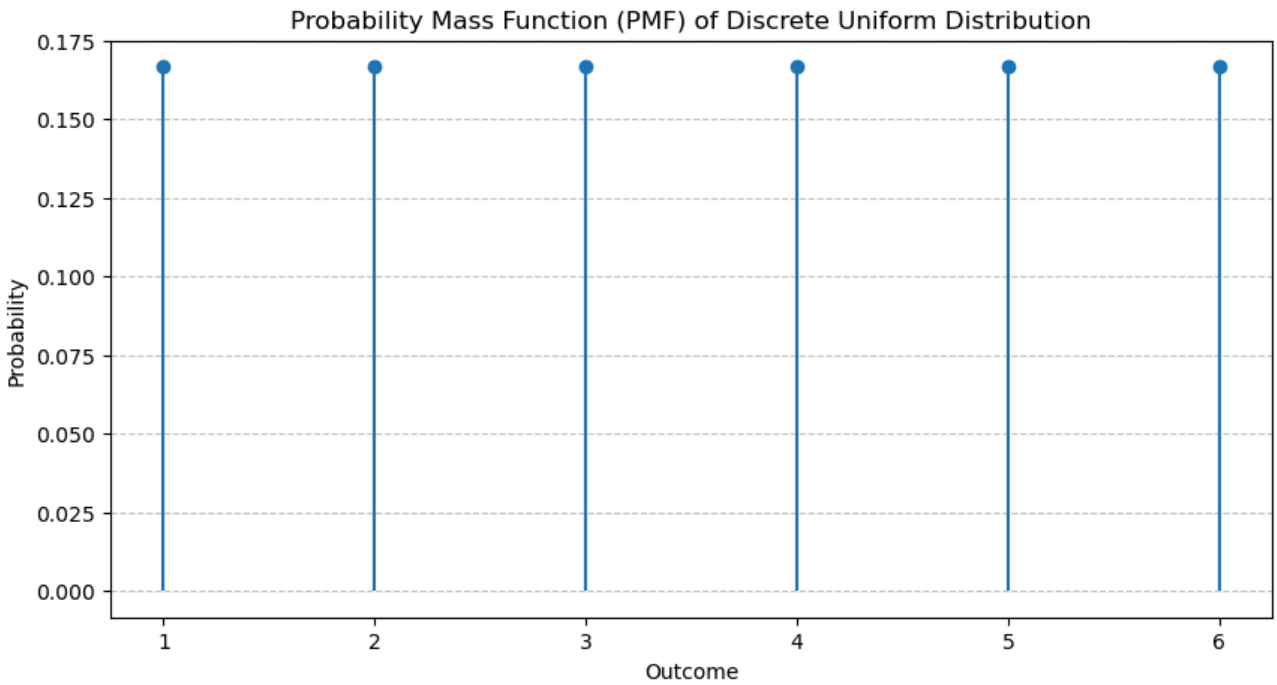
Where:

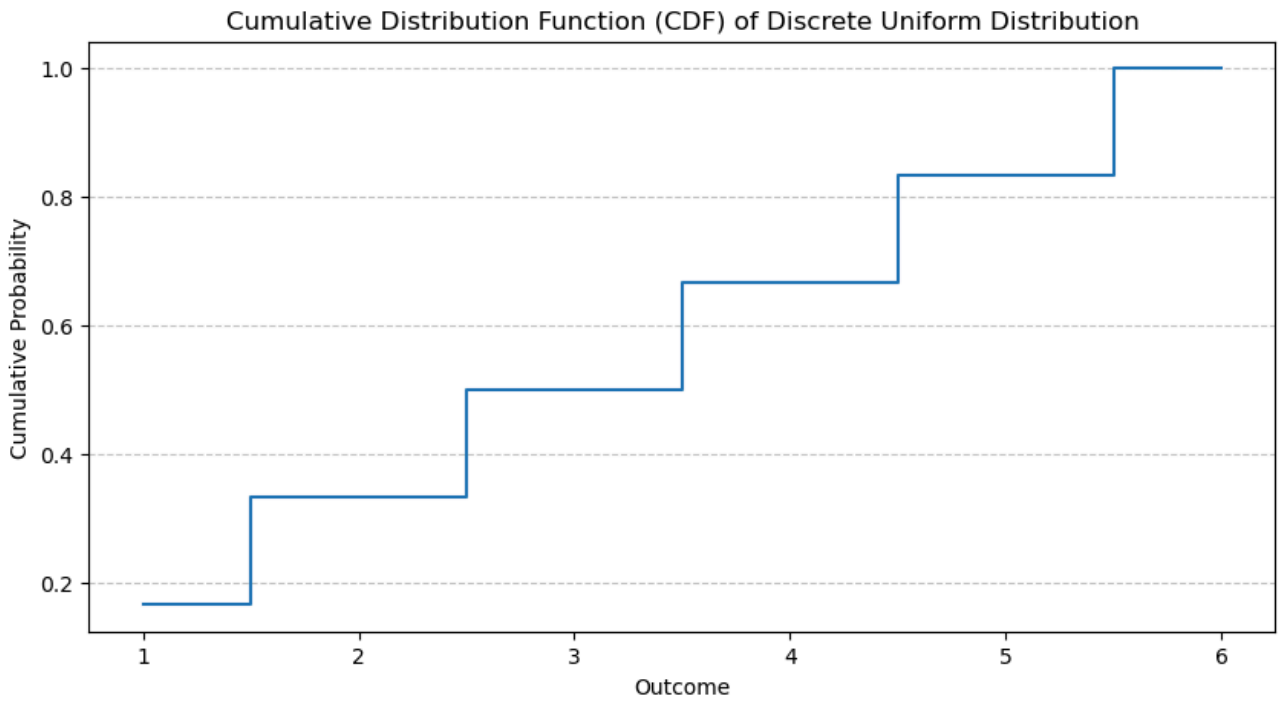
- $F(x)$ is the cumulative probability that X is less than or equal to x.
- The CDF accumulates the probabilities of all outcomes up to x.

The Discrete Uniform distribution is commonly used in scenarios such as rolling a fair six-sided die or selecting a random number from a range of integers. Here's an example of using the Discrete Uniform distribution to model the outcome of rolling a fair six-sided die

In [21]:
```
import numpy as np
import matplotlib.pyplot as plt

# Define the parameters
a = 1  # Minimum value of the range
b = 6  # Maximum value of the range
num_outcomes = b - a + 1  # Number of possible outcomes

# Define the values of the random variable X
x_values = np.arange(a, b+1)

# Calculate the probability mass function (PMF)
pmf = np.ones_like(x_values) / num_outcomes

# Calculate the cumulative distribution function (CDF)
cdf = np.cumsum(pmf)

# Plot the PMF
```

```python
plt.figure(figsize=(10, 5))
plt.stem(x_values, pmf, markerfmt='o', basefmt=" ", linefmt='-')
plt.title('Probability Mass Function (PMF) of Discrete Uniform Distribution')
plt.xlabel('Outcome')
plt.ylabel('Probability')
plt.xticks(x_values)
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.show()

# Plot the CDF
plt.figure(figsize=(10, 5))
plt.step(x_values, cdf, where='mid')
plt.title('Cumulative Distribution Function (CDF) of Discrete Uniform Distribution')
plt.xlabel('Outcome')
plt.ylabel('Cumulative Probability')
plt.xticks(x_values)
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.show()
```





Bernoulli Distribution

The Bernoulli distribution represents the probability distribution of a random variable that takes on only two possible outcomes, typically labeled as success (1) and failure (0), with a probability of success p and failure $1-p$ in a single trial.

Probability Mass Function (PMF)

The probability mass function (PMF) of a Bernoulli random variable X is given by:

$$P(X=x) = \begin{cases} 1-p & \text{if } x=0 \\ p & \text{if } x=1 \end{cases}$$

Where:

- p is the probability of success (taking the value 1).
- $1-p$ is the probability of failure (taking the value 0).
- The PMF specifies the probability of each possible outcome.

Cumulative Distribution Function (CDF)

The cumulative distribution function (CDF) of a Bernoulli random variable X is given by:

$$F(x) = \begin{cases} 0 & \text{if } x<0 \\ 1-p & \text{if } 0 \leq x < 1 \\ 1 & \text{if } x \geq 1 \end{cases}$$

The Bernoulli distribution is often used to model simple binary outcomes, such as the success or failure of an event, the presence or absence of a characteristic, or the outcome of a coin toss.

Example: Tossing an unfair coin

Let's consider the scenario of tossing an unfair coin, where the probability of getting heads is 0.3 and the probability of getting tails is 0.7.

Random Variable Definition

Our sample space is $S = \{0, 1\}$. We define the random variable $X : S \to \mathbb{R}$, where $X(s) = s$ as follows:

- $X(1) = 1$ if the outcome is heads (success).
- $X(0) = 0$ if the outcome is tails (failure).

This random variable always follows a Bernuilli distribution.

Probability Mass Function (PMF)

The probability mass function (PMF) of X for this unfair coin toss is given by:

$$P(X = x) = \begin{cases} 0.7 & \text{if } x = 0 \\ 0.3 & \text{if } x = 1 \end{cases}$$

Cumulative Distribution Function (CDF)

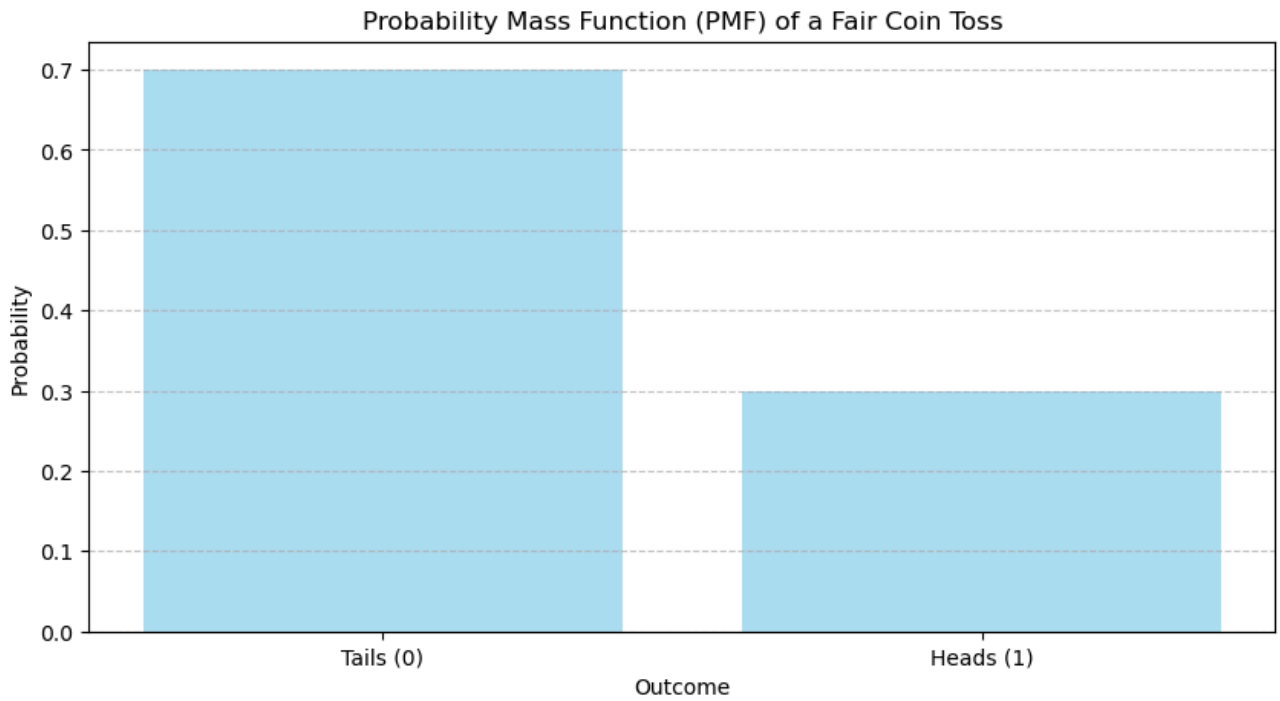
The cumulative distribution function (CDF) of X is calculated as follows:

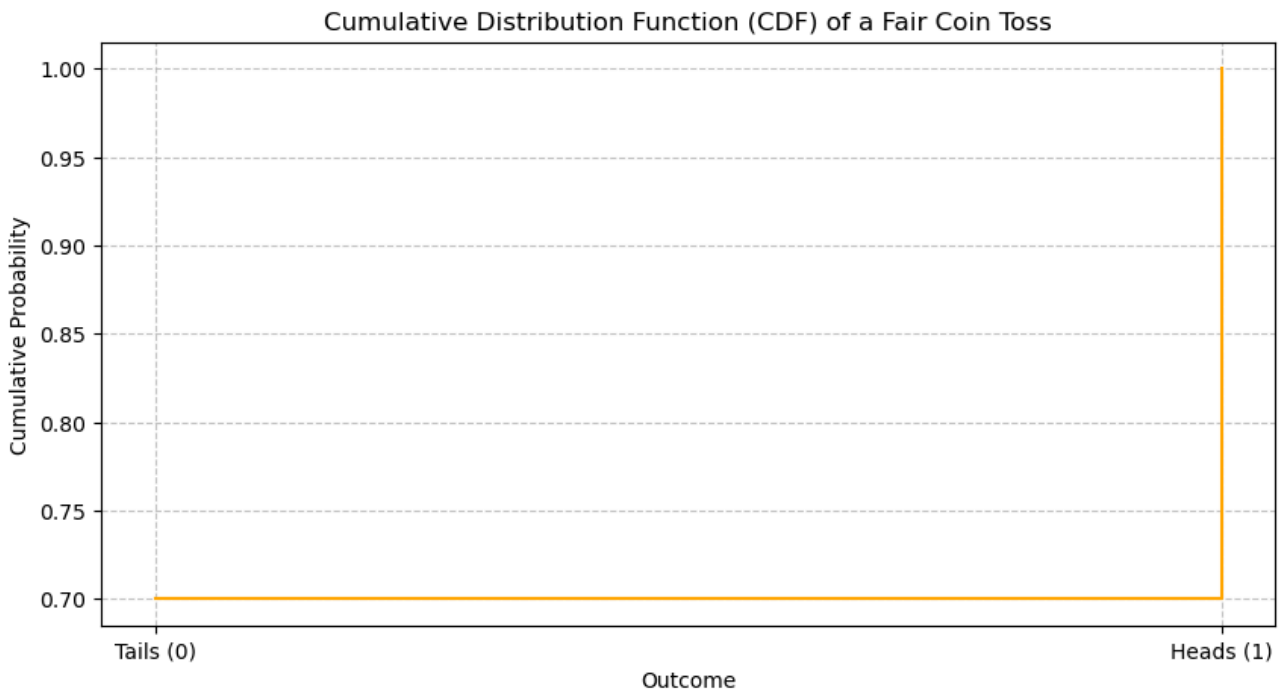
$$F(x) = \begin{cases} 0 & \text{if } x < 0 \\ 0.7 & \text{if } 0 \leq x < 1 \\ 1 & \text{if } x \geq 1 \end{cases}$$

Now, let's visualize the PMF and CDF for this unfair coin toss.

In [22]:
```
import numpy as np
import matplotlib.pyplot as plt

# Define the parameters
p = 0.3  # Probability of success (getting heads)

# Define the sample space and outcomes
sample_space = np.array([0, 1])  # 0 represents tails, 1 represents heads

# Define the PMF
pmf = [1-p, p]

# Calculate the CDF
cdf = np.cumsum(pmf)

# Plot the PMF
plt.figure(figsize=(10, 5))
plt.bar(sample_space, pmf, color='skyblue', alpha=0.7)
plt.title('Probability Mass Function (PMF) of a Fair Coin Toss')
plt.xlabel('Outcome')
plt.ylabel('Probability')
plt.xticks(sample_space, ['Tails (0)', 'Heads (1)'])
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.show()
```

```python
# Plot the CDF
plt.figure(figsize=(10, 5))
plt.step(sample_space, cdf, color='orange', where='post')
plt.title('Cumulative Distribution Function (CDF) of a Fair Coin Toss')
plt.xlabel('Outcome')
plt.ylabel('Cumulative Probability')
plt.xticks(sample_space, ['Tails (0)', 'Heads (1)'])
plt.grid(axis='both', linestyle='--', alpha=0.7)
plt.show()
```





Binomial Distribution

The Binomial distribution describes the probability distribution of the number of successes in a fixed number of binary trials, each with the same probability of success p. It is characterized by two

parameters: the number of trials n and the probability of success p.

Probability Mass Function (PMF)

The probability mass function (PMF) of a Binomial random variable X is given by:

$$P(X = k) = \binom{n}{k} p^k (1-p)^{n-k}$$

Where:

- k is the number of successes (outcomes of interest).
- n is the total number of trials.
- p is the probability of success in each trial.
- $\binom{n}{k}$ represents the binomial coefficient, calculated as $\frac{n!}{k!(n-k)!}$.

The PMF specifies the probability of getting exactly k successes in n trials.

Cumulative Distribution Function (CDF)

The cumulative distribution function (CDF) of a Binomial random variable X is calculated by summing the probabilities of all outcomes up to k:

$$F(k) = \sum_{i=0}^{k} \binom{n}{i} p^i (1-p)^{n-i}$$

Where:

- $F(k)$ is the cumulative probability that X is less than or equal to k.
- The sum is taken over all possible values of i from 0 to k.

The CDF accumulates the probabilities of getting i or fewer successes in n trials.

The Binomial distribution is commonly used to model the number of successes in repeated independent Bernoulli trials, such as the number of heads in multiple coin tosses or the number of successes in multiple independent experiments.

Example: Throwing a dice 10 times

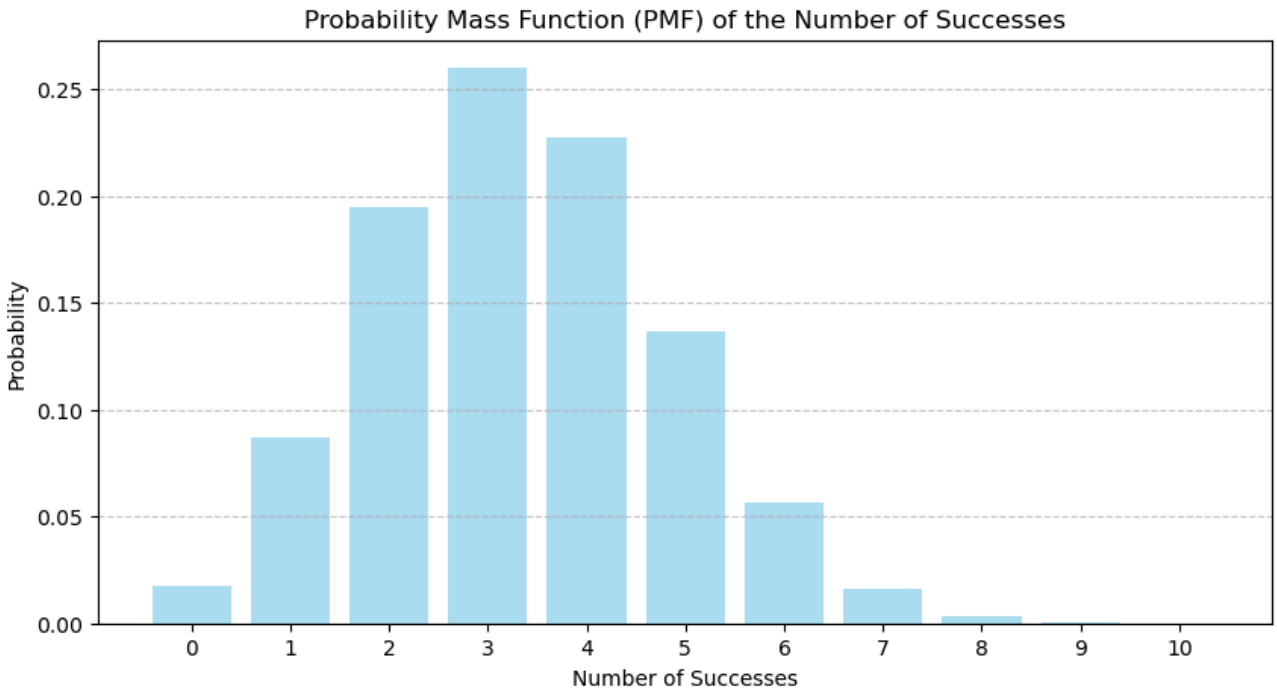
Let's create an example for throwing 10 fair six-sided die, where success (in any particular dice) is defined as rolling a number greater than 4. The sample space S would be the set of all possible outcomes of 10 throws. The random variable defining this experiment is $X : S \to \mathbb{R}$, where $X(s)$ is the number of sucesses for the trow s.

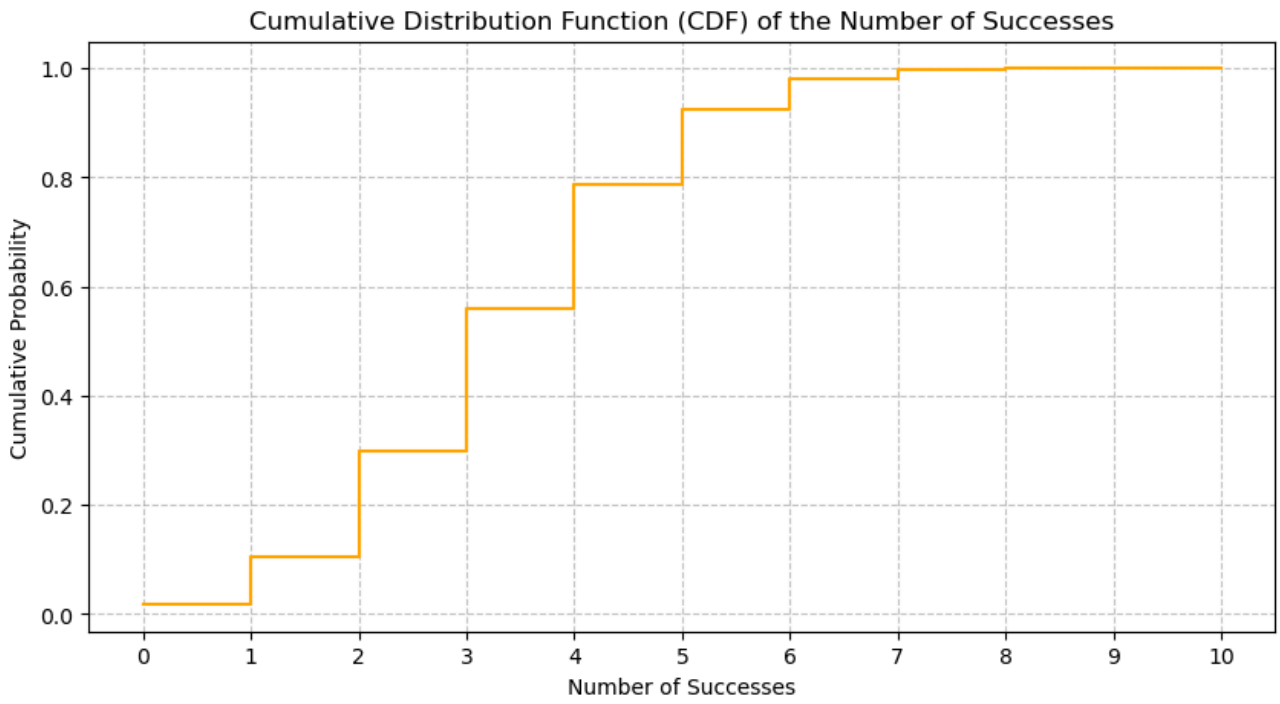
If we now want to compute the probability of getting four sucesses:

$$P(X = 4) = \binom{10}{4} (1/3)^4 (2/3)^6$$

In [23]:

```python
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import binom

# Define the parameters
n = 10  # Number of trials (throwing the die 6 times)
p = 1/3  # Probability of success (rolling a number greater than 4)

# Define the sample space (number of successes)
successes = np.arange(0, n+1)

# Calculate the PMF
pmf = binom.pmf(successes, n, p)

# Calculate the CDF
cdf = binom.cdf(successes, n, p)

# Plot the PMF
plt.figure(figsize=(10, 5))
plt.bar(successes, pmf, color='skyblue', alpha=0.7)
plt.title('Probability Mass Function (PMF) of the Number of Successes')
plt.xlabel('Number of Successes')
plt.ylabel('Probability')
plt.xticks(successes)
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.show()

# Plot the CDF
plt.figure(figsize=(10, 5))
plt.step(successes, cdf, color='orange', where='post')
plt.title('Cumulative Distribution Function (CDF) of the Number of Successes')
plt.xlabel('Number of Successes')
plt.ylabel('Cumulative Probability')
plt.xticks(successes)
plt.grid(axis='both', linestyle='--', alpha=0.7)
plt.show()
```





Exercise 1:

You are playing a game where you roll a ten-sided die 20 times. You win the game if you get more than 2 numbers greater than 8. What is the probability of winning the game?

In [24]:
```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import binom

# Define the parameters
n = 20  # Number of trials (rolling the ten-sided die 20 times)
p = 2/10  # Probability of success (rolling a number greater than 8)
k = 2  # Number of successes we're interested in (more than 2 numbers greater than 8)

# Calculate the probability of winning the game
prob_win = 1 - binom.cdf(k, n, p)

# Print the probability of winning the game
print("Probability of winning the game:", prob_win)
```

```
Probability of winning the game: 0.7939152810515259
```

Poisson Distribution

The Poisson distribution models the number of events occurring in a fixed interval of time or space, given a known average rate of occurrence λ. It is often used to describe rare events with a low probability of occurrence.

Probability Mass Function (PMF)

The probability mass function (PMF) of a Poisson random variable X is given by:

$$P(X = k) = \frac{e^{-\lambda}\lambda^k}{k!}$$

Where:

- k is the number of events (outcomes of interest).
- λ is the average rate of occurrence of events.
- e is the base of the natural logarithm, approximately equal to 2.71828.
- $k!$ represents the factorial of k.

The PMF specifies the probability of observing exactly k events in the given interval.

Cumulative Distribution Function (CDF)

The cumulative distribution function (CDF) of a Poisson random variable X is given by:

$$F(k) = \sum_{i=0}^{k} \frac{e^{-\lambda}\lambda^i}{i!}$$

Where:

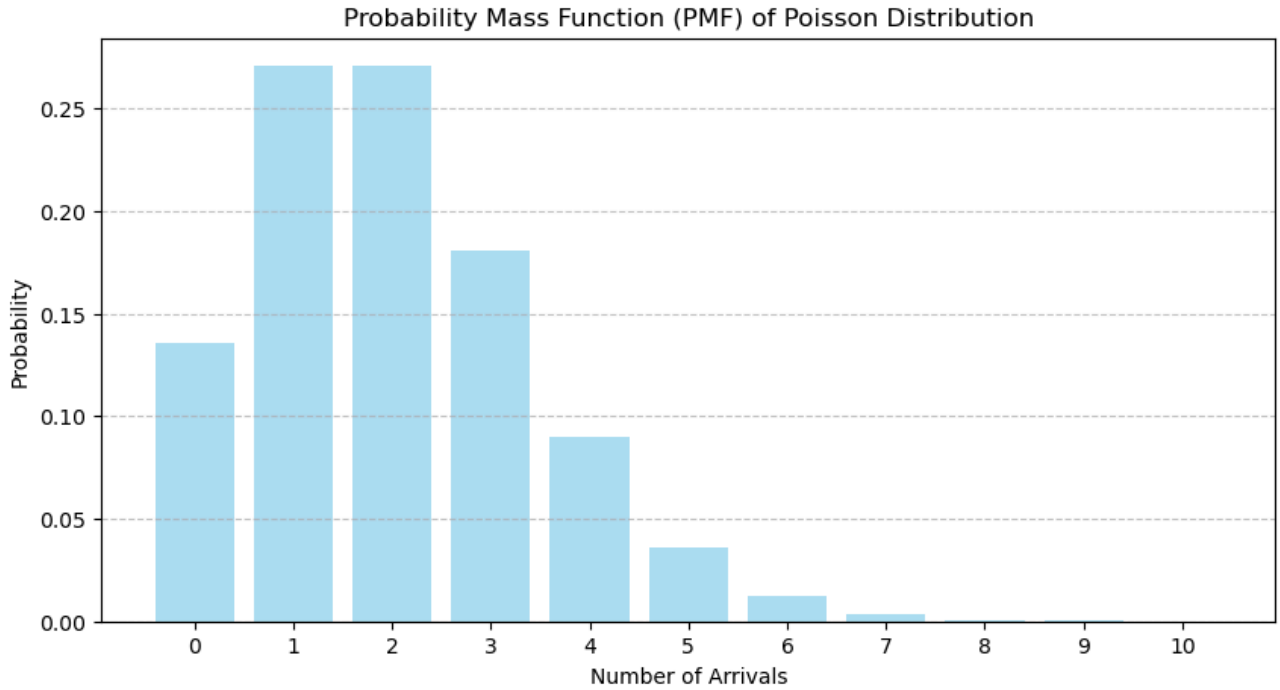
- $F(k)$ is the cumulative probability that X is less than or equal to k.
- The sum is taken over all possible values of i from 0 to k.

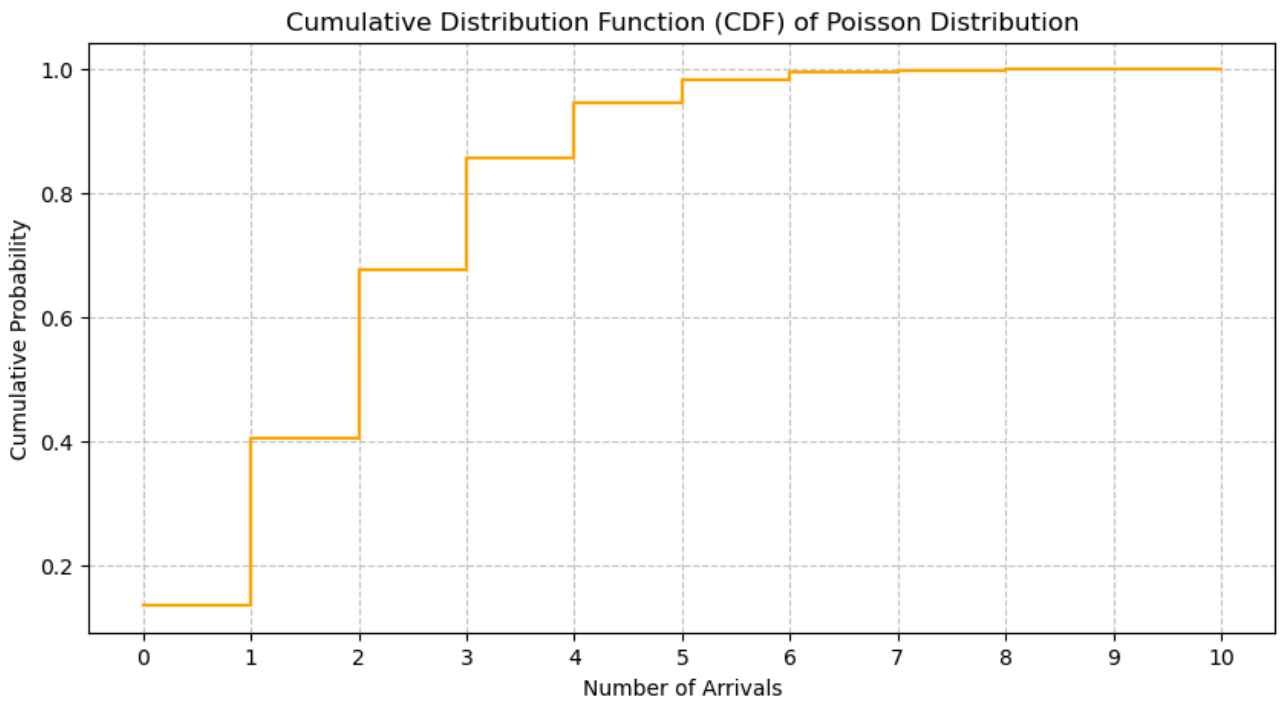
The CDF accumulates the probabilities of observing i or fewer events in the given interval.

The Poisson distribution is commonly used to model the number of arrivals in a fixed time period, such as the number of phone calls received in a call center per hour or the number of customers arriving at a store per day. Here's an example of using the Poisson distribution to model the number of customers arriving at a store per hour.

In [25]:
```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import poisson

# Define the parameters
lambd = 2  # Average rate of arrival (customers per hour)
k_values = np.arange(0, 11)  # Number of arrivals (from 0 to 10)

# Calculate the PMF
pmf = poisson.pmf(k_values, mu=lambd)

# Calculate the CDF
cdf = poisson.cdf(k_values, mu=lambd)

# Plot the PMF
plt.figure(figsize=(10, 5))
plt.bar(k_values, pmf, color='skyblue', alpha=0.7)
plt.title('Probability Mass Function (PMF) of Poisson Distribution')
plt.xlabel('Number of Arrivals')
plt.ylabel('Probability')
plt.xticks(k_values)
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.show()

# Plot the CDF
```

```
plt.figure(figsize=(10, 5))
plt.step(k_values, cdf, color='orange', where='post')
plt.title('Cumulative Distribution Function (CDF) of Poisson Distribution')
plt.xlabel('Number of Arrivals')
plt.ylabel('Cumulative Probability')
plt.xticks(k_values)
plt.grid(axis='both', linestyle='--', alpha=0.7)
plt.show()
```





Geometric Distribution

The Geometric distribution represents the number of trials needed to achieve the first success in a sequence of independent trials, each with the same probability of success p. It is characterized by a single parameter: the probability of success p.

Probability Mass Function (PMF)

The probability mass function (PMF) of a Geometric random variable X is given by:

$$P(X = k) = (1 - p)^{k-1}p$$

Where:

- k is the number of trials needed to achieve the first success.
- p is the probability of success in each trial.

The PMF specifies the probability of needing exactly k trials to achieve the first success.

Cumulative Distribution Function (CDF)

The cumulative distribution function (CDF) of a Geometric random variable X is given by:

$$F(k) = 1 - (1 - p)^k$$

Where:

- $F(k)$ is the cumulative probability that X is less than or equal to k.

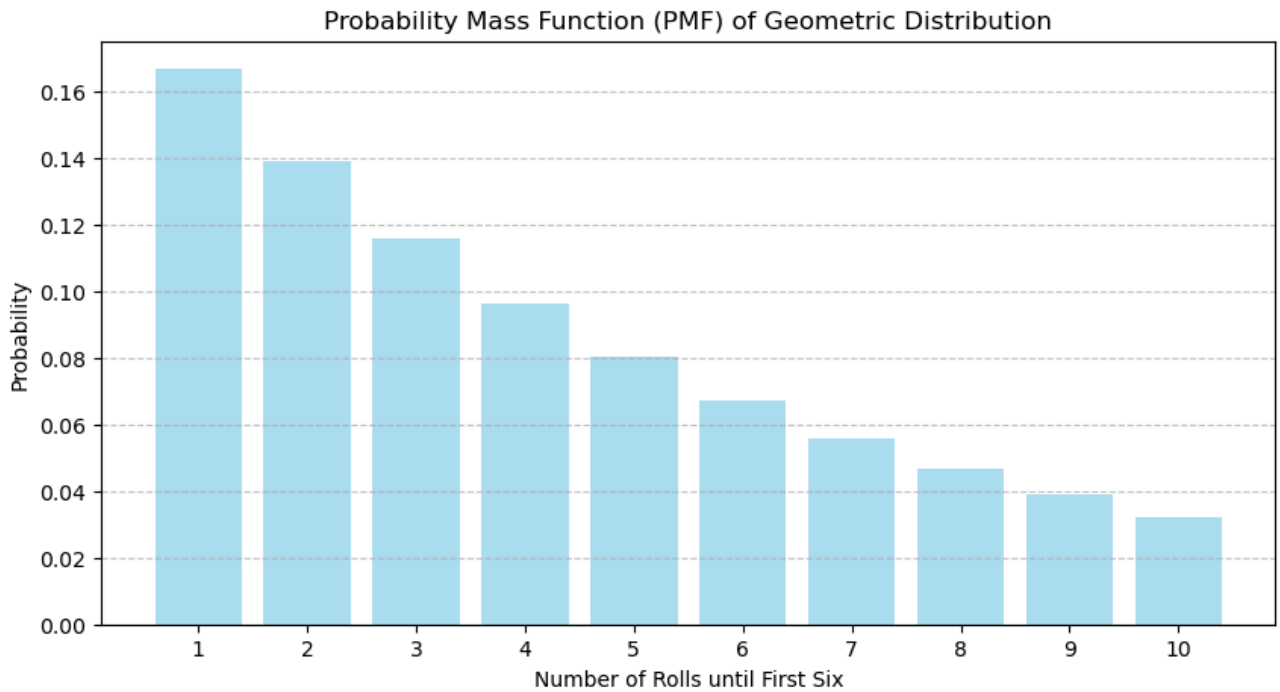
The CDF accumulates the probabilities of needing k or fewer trials to achieve the first success.

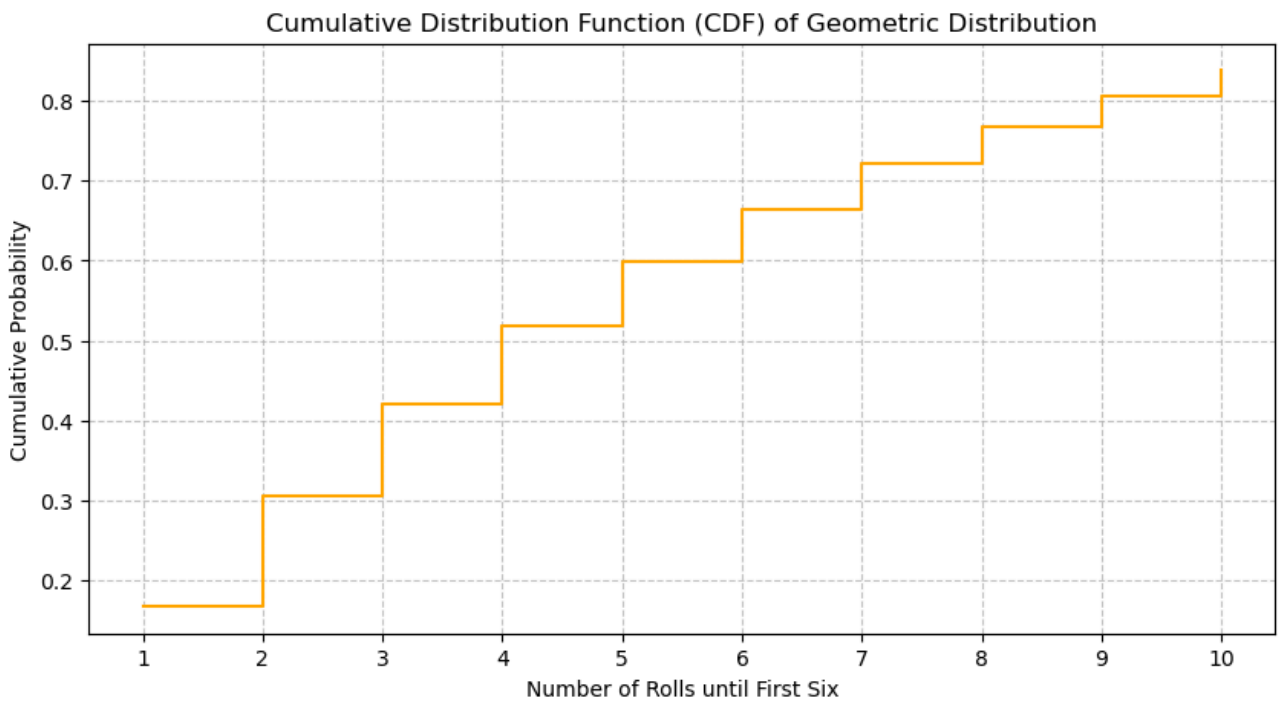
The Geometric distribution is commonly used to model the number of trials needed until the first success in scenarios such as what is the probability of needing N trials until I roll a 6 on a six sided dice.

In [26]:

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import geom

# Define the parameter
p = 1/6  # Probability of success (rolling a six on a fair six-sided die)

# Define the values of k (number of trials)
k_values = np.arange(1, 11)  # Number of trials from 1 to 10

# Calculate the PMF
pmf = geom.pmf(k_values, p)

# Calculate the CDF
cdf = geom.cdf(k_values, p)

# Plot the PMF
plt.figure(figsize=(10, 5))
plt.bar(k_values, pmf, color='skyblue', alpha=0.7)
plt.title('Probability Mass Function (PMF) of Geometric Distribution')
plt.xlabel('Number of Rolls until First Six')
plt.ylabel('Probability')
plt.xticks(k_values)
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.show()
```

```python
# Plot the CDF
plt.figure(figsize=(10, 5))
plt.step(k_values, cdf, color='orange', where='post')
plt.title('Cumulative Distribution Function (CDF) of Geometric Distribution')
plt.xlabel('Number of Rolls until First Six')
plt.ylabel('Cumulative Probability')
plt.xticks(k_values)
plt.grid(axis='both', linestyle='--', alpha=0.7)
plt.show()
```





Montecarlo Method

Introduction to Monte Carlo Method

The Monte Carlo method is a powerful computational technique used to solve mathematical problems through statistical sampling. It provides a systematic way to approximate complex systems or compute numerical solutions for problems that are difficult to solve analytically. The method derives its name from the famous MonteCarlo Casino in Monaco, known for its games of chance and randomness, reflecting the stochastic nature of the method.

Monte Carlo methods rely on generating random samples from known probability distributions to estimate numerical quantities or simulate the behavior of systems. By repeatedly sampling from these distributions and aggregating the results, Monte Carlo simulations can provide accurate approximations of quantities of interest, such as expected values, probabilities, or the behavior of complex systems.

Definition of Monte Carlo Method

The Monte Carlo method is a computational technique based on random sampling to approximate numerical solutions for problems that involve uncertainty or complexity. It involves the following steps:

1. **Modeling**: Formulate the problem in a mathematical framework, defining the inputs, parameters, and constraints.
2. **Sampling**: Generate random samples from relevant probability distributions to represent uncertain or variable quantities in the problem.
3. **Analysis**: Analyze the results of the simulations to draw conclusions about the system or solve the problem of interest.

Example: The owner

You own two stores, each with its own customer arrival rate. In the first store, customers arrive at an average rate of 2 customers per hour, while in the second store, customers arrive at an average rate of 3 customers per hour. You want to know the probability of having more than 5 customers between the two stores within the next hour.

Modeling the Problem:

We can model the number of customers arriving at each store using Poisson distributions, as the Poisson distribution is commonly used to model the number of arrivals in a fixed time period. For the first store, the parameter λ_1 is 2 (average rate of arrival), and for the second store, the parameter λ_2 is 3.

Solution Approach:

1. **Monte Carlo Simulation**: We'll simulate multiple scenarios by repeatedly sampling from the Poisson distributions for each store to estimate the total number of customers arriving at both stores in the next hour.

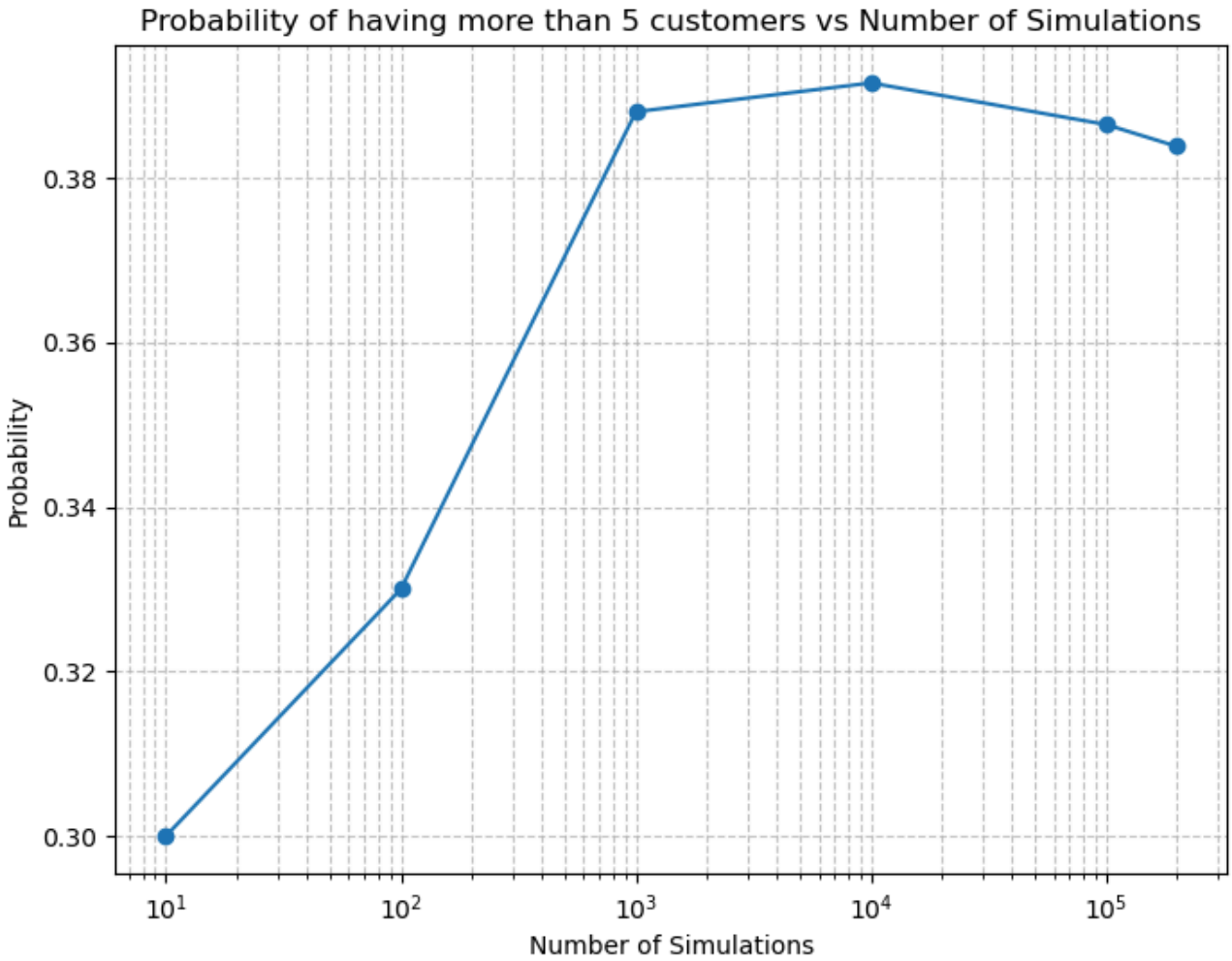
2. **Counting**: We'll count the number of simulations where the total number of customers is more than 5.

3. **Probability Calculation**: Finally, we'll calculate the probability of having more than 5 customers between the two stores by dividing the count of favorable outcomes by the total number of simulations.

Let's proceed to implement this solution using Monte Carlo simulation.

In [27]:
```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import poisson

# Parameters
lambda_1 = 2  # Average rate of arrival at the first store
lambda_2 = 3  # Average rate of arrival at the second store
num_simulations_list = [10, 100, 1000, 10000, 100000,200000]  # Number of Monte Carlo si

# Initialize list to store probabilities
probabilities = []

# Perform Monte Carlo simulation for each number of simulations
for num_simulations in num_simulations_list:
    # Initialize count of favorable outcomes
    favorable_outcomes = 0

    # Perform Monte Carlo simulation
    for _ in range(num_simulations):
        # Generate random samples from Poisson distributions for each store
        num_customers_store1 = poisson.rvs(lambda_1)
        num_customers_store2 = poisson.rvs(lambda_2)

        # Calculate total number of customers arriving at both stores
        total_customers = num_customers_store1 + num_customers_store2

        # Check if total number of customers is more than 5
        if total_customers > 5:
            favorable_outcomes += 1

    # Calculate probability of having more than 5 customers
    probability_more_than_5_customers = favorable_outcomes / num_simulations

    # Append probability to the list
    probabilities.append(probability_more_than_5_customers)

    print("Probability with", num_simulations, "simulations:", probability_more_than_5_c

# Plot probabilities
plt.figure(figsize=(8, 6))
plt.plot(num_simulations_list, probabilities, marker='o', linestyle='-')
plt.xscale('log')
plt.title('Probability of having more than 5 customers vs Number of Simulations')
plt.xlabel('Number of Simulations')
plt.ylabel('Probability')
plt.grid(True, which="both", ls="--", alpha=0.7)
plt.show()
```

```
Probability with 10 simulations: 0.3
Probability with 100 simulations: 0.33
Probability with 1000 simulations: 0.388
Probability with 10000 simulations: 0.3915
Probability with 100000 simulations: 0.38645
Probability with 200000 simulations: 0.3838
```



Example: Alistair is back

You are playing Dungeons and Dragons with some of your friends (yes, again). In the game, you are the valiant knight Sir Alistair Elthond. As Sir Alistair faced off against the Orc Champion in the heat of battle, their swords clashed with thunderous force, echoing across the battlefield. ... With each swing, Alistair's determination burned bright, fueled by the need to protect his comrades and uphold the honor of his knightly order. Yet, despite his skill and valor, the Orc Champion proved to be a formidable adversary, matching his every move with brutal strength. As the duel reached its climax, the Orc Champion, fueled by rage and ferocity, raised his massive weapon high overhead, poised to deliver a terrifying strike. With a steely gaze and great resolve, Sir Alistair maved to avoid the strike, which left the Orc Champion wide open for a fatal blow. ... Alistair wields the Sword Of Answering, giving him a +3 on attack and causing 1D10 + 1D4 vital points (that is a 10-sided die plus a four sided die). If Alistair needs a 14 to hit the Orc Champion (a 20-sided die is thrown for this) and he only has 13 vital points left. What is the probability of Alistair ending the misserable life of this evil champion?

If Alistair fails his attack or do not kill his opponent, the Orc Champion will land his fatal blow. He needs a 12 to hit Alistair to cause him 3D6 vital points of damage (that is the sum of three six-sided dice). However, our hero is already wounded and only has 3 vital points left. What is the probability of the Orc winning the combat? **Solve it using Montecarlo**

In [28]:
```
import numpy as np

# Define the parameters
num_simulations = 100000  # Number of Monte Carlo simulations

# Function to simulate combat between Sir Alistair and the Orc Champion
def simulate_combat():
    # Sir Alistair's attack
    attack_roll = np.random.randint(1, 21) + 3  # 20-sided die + 3 bonus
    damage_roll = np.random.randint(1, 11) + np.random.randint(1, 5)  # 10-sided die + 4
    if attack_roll >= 14 and damage_roll>12:  # Alistair hits the Orc Champion
        return "Alistair wins", damage_roll
    else:
        # Orc Champion's attack
        attack_roll = np.random.randint(1, 21)
        if attack_roll >= 12:  # Orc Champion hits Alistair
            return "Orc wins", damage_roll
        else:
            return "Draw", 0

# Initialize counters for outcomes
alistair_wins = 0
orc_wins = 0

# Simulate combat for each iteration
for _ in range(num_simulations):
    outcome, damage = simulate_combat()
    if outcome == "Alistair wins":
        alistair_wins += 1
    elif outcome == "Orc wins":
        orc_wins += 1

# Calculate probabilities
probability_alistair_wins = alistair_wins / num_simulations
probability_orc_wins = orc_wins / num_simulations

print("Probability of Sir Alistair winning the combat:", probability_alistair_wins)
print("Probability of the Orc Champion winning the combat:", probability_orc_wins)
```

```
Probability of Sir Alistair winning the combat: 0.03735
Probability of the Orc Champion winning the combat: 0.43131
```

In [ ]: